

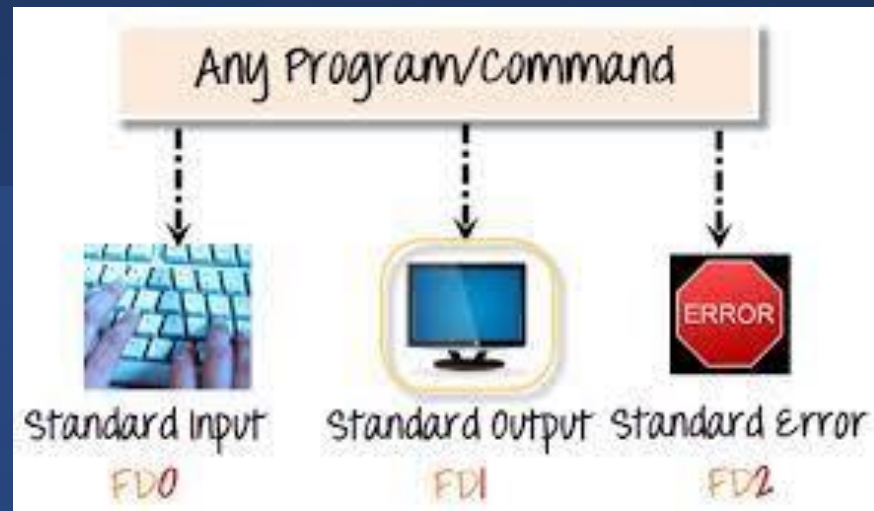


Higher Diploma in Computer Science

Computer Systems & Networks



Input/Output Redirection



You might consider using redirection to:

- Redirect a command output to a file, instead of to the monitor
- Redirect a program (that by default receives input from the keyboard) to instead receive input from a file
- Redirect error messages to files, instead of to the screen by default

File Descriptors (FD)

- The kernel acts as **an interface between your hardware devices, constantly moving data between i/p and o/p devices**
- In Linux, a process is **any active (running) instance of a program.**
 - a program is any executable file held in storage on your machine.
 - Anytime you run a program, you have created a process
- Processes know where data should be sent to using **file descriptors.**

File Descriptors (FD)

File	FD
STDIN	0
STDOUT	1
STDERR	2

- In Linux/Unix, **everything** is a file
- Every File has an associated number called File Descriptor (**FD**).
- If a program is executed the output is sent to the FD of the screen (FD 1) & you see program output on your monitor (default).
- Similarly, if the output is sent to FD of the printer, the program output would have been printed.

Look at the file descriptors of given processes

- Devices registered when the kernel was instantiated can be seen in the **/dev** directory of your host.

File	FD
STDIN	0
STDOUT	1
STDERR	2

```
compsys@compsys-virtualbox:~$ ls -l /dev/ | less
```

- Can you see their FD's, as expected

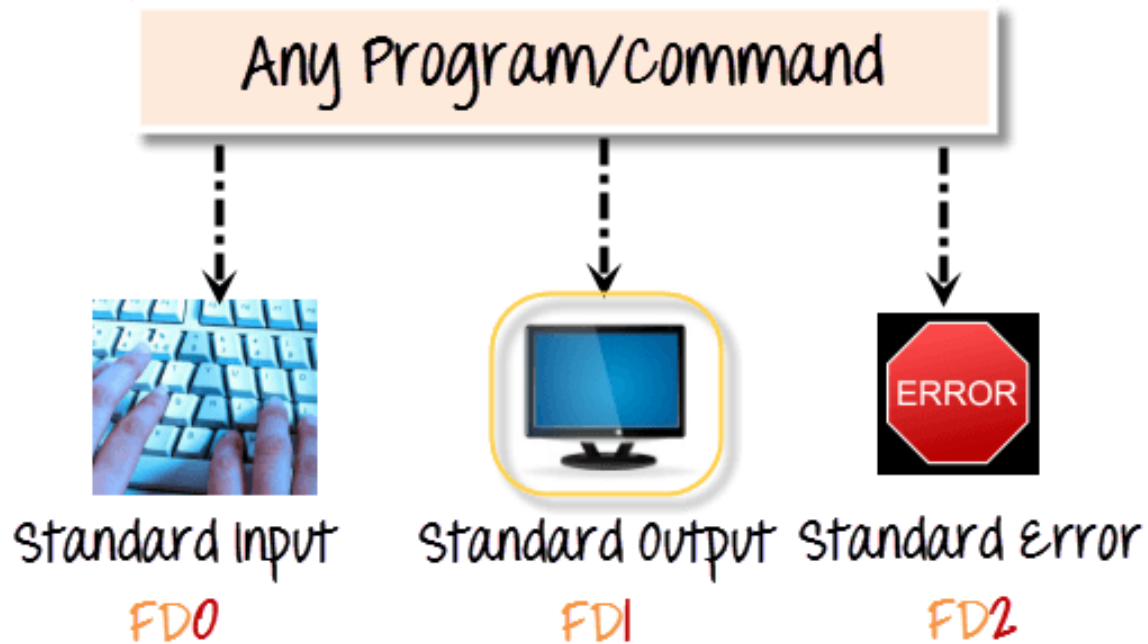
```
lrwxrwxrwx 1 root root 15 Sep 2 12:45 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root 15 Sep 2 12:45 stdin -> /proc/self/fd/0
lrwxrwxrwx 1 root root 15 Sep 2 12:45 stdout -> /proc/self/fd/1
```



- ...allows for command line information to be passed to different *streams*
- The streams are:
 - STDIN (0) - Standard input (default keyboard)
 - STDOUT (1) - Standard output (data printed by the program, defaults to the screen)
 - STDERR (2) - Standard error (for error messages, also defaults to the screen)

Input/Output Redirection

- 3 files open when you exe any program/command:



File	FD
STDIN	0
STDOUT	1
STDERR	2

- redirection=routing defaults to something other than the default

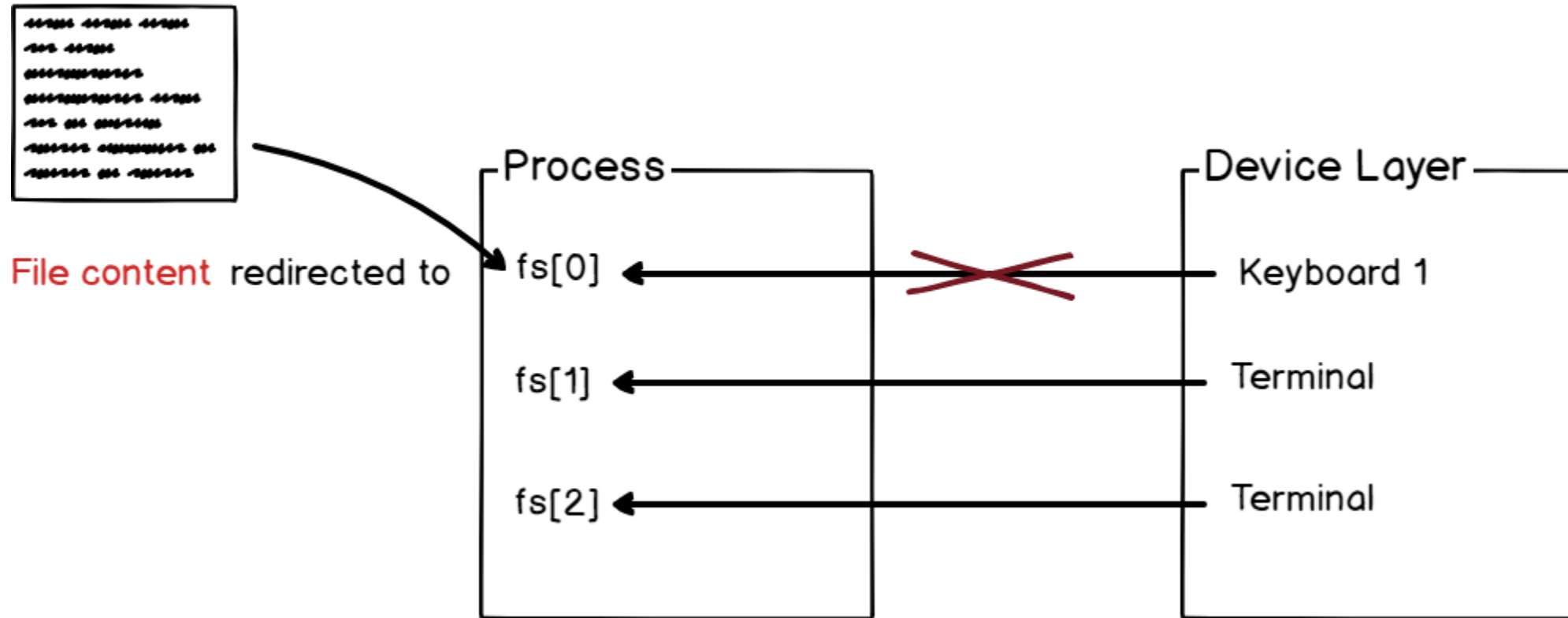
Redirection Metacharacters

Command redirection is enabled by the following standard shell **metacharacters**:

- Redirection of standard input (<)
- Redirection of standard output (>)
- Redirection of standard error (2>)
- The pipe character (|)

Redirection of standard input <

Input redirection on Linux



Redirection of standard input <

File	FD
STDIN	0
STDOUT	1
STDERR	2

\$command < filename

or

\$command 0< filename

Redirection of standard input <

```
compsys@compsys-virtualbox:~$ cat emp*
300 frank manager sales €5000
400 caroline developer technology €5500
100 mary sysadmin technology €7000
200 rosanne manager marketing €9500
500 eamonn dba technology €6000
600 colm sysadmin technology €4500
```

```
compsys@compsys-virtualbox:~$ sort < emp*
100 mary sysadmin technology €7000
200 rosanne manager marketing €9500
300 frank manager sales €5000
400 caroline developer technology €5500
500 eamonn dba technology €6000
600 colm sysadmin technology €4500
compsys@compsys-virtualbox:~$ cat emp*
300 frank manager sales €5000
400 caroline developer technology €5500
100 mary sysadmin technology €7000
200 rosanne manager marketing €9500
500 eamonn dba technology €6000
600 colm sysadmin technology €4500
```

Redirection of standard input <

File	FD
STDIN	0
STDOUT	1
STDERR	2

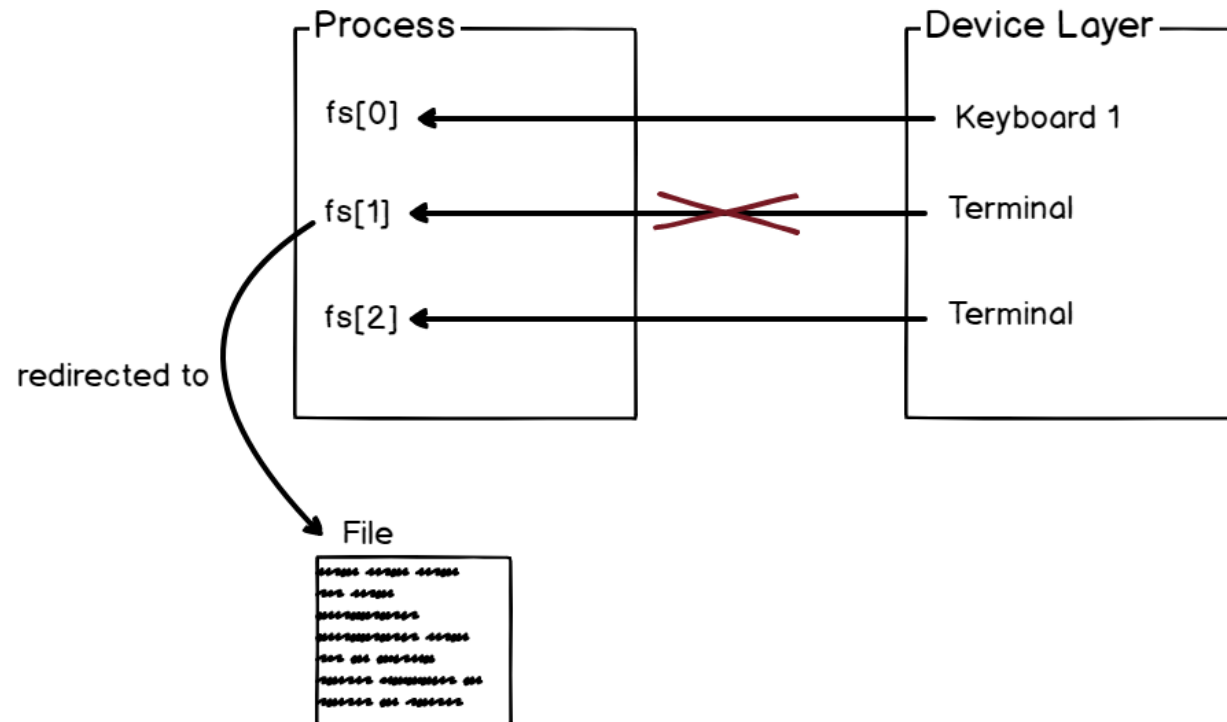
The commands that normally take their input from the standard input can have their input redirected from a file in this manner.

E.G. to count the number of lines in a file named *stats*, you can execute the command

```
$wc -l < stats
```

Redirection of standard output (>)

Output redirection on Linux



Redirection of standard output (>)

File	FD
STDIN	0
STDOUT	1
STDERR	2

- The standard output STDOUT of the `echo` command is to the screen:

```
compsys@compsys-virtualbox:~$ echo "Caroline Cahill"  
Caroline Cahill
```

- Redirected the standard output to a file "classlist.txt" instead:

```
$ echo "Caroline Cahill" > classlist.txt
```

- View contents of classlist.txt file:

```
$ cat classlist.txt  
Caroline Cahill
```

Redirection of standard output (>)

PROBLEM:

Redirecting using > will **overwrite**



```
$ hostname > classlist.txt
```

`cat` the new contents of `classlist.txt`

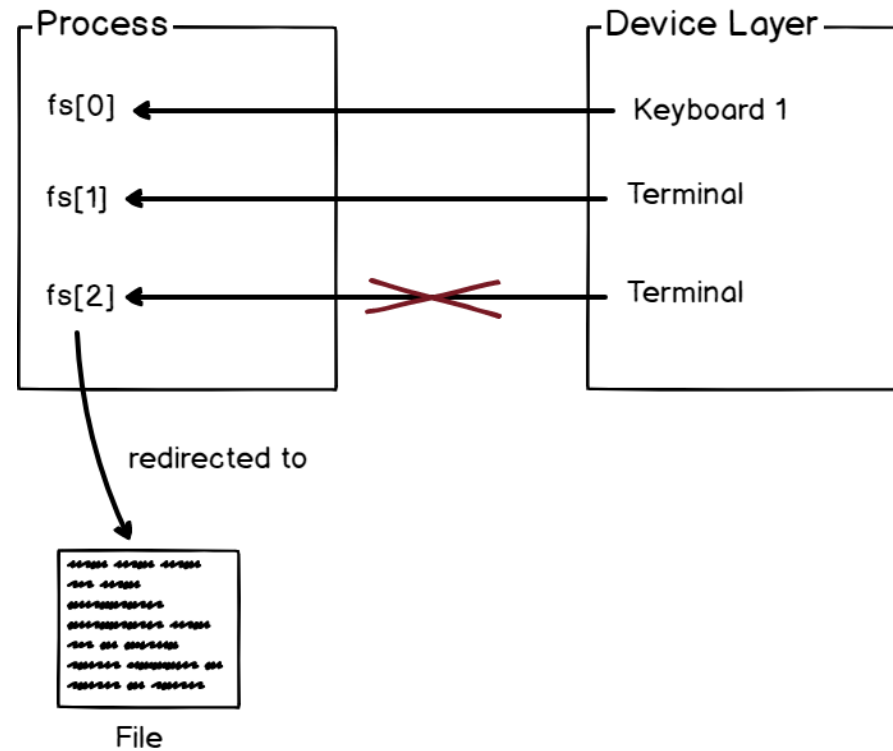
Redirection of standard output (>>)

- Preserve the contents of an existing file using the >> characters
 - append to a file instead of overwriting it.

```
compsys@compsys-virtualbox:~$ date > classlist.txt
compsys@compsys-virtualbox:~$ cat class*
Fri 02 Sep 2022 12:30:46 IST
compsys@compsys-virtualbox:~$ hostname >> classli*
compsys@compsys-virtualbox:~$ cat class*
Fri 02 Sep 2022 12:30:46 IST
compsys-virtualbox
compsys@compsys-virtualbox:~$ uname -r >> classli*
compsys@compsys-virtualbox:~$ cat class*
Fri 02 Sep 2022 12:30:46 IST
compsys-virtualbox
5.15.0-47-generic
compsys@compsys-virtualbox:~$ echo "Phew, each line was appended to the end" >> classli*
compsys@compsys-virtualbox:~$ cat class*
Fri 02 Sep 2022 12:30:46 IST
compsys-virtualbox
5.15.0-47-generic
Phew, each line was appended to the end
compsys@compsys-virtualbox:~$ █
```

Redirection of standard error (2>)

Error redirection on Linux



Redirection of standard error (2>)

File	FD
STDIN	0
STDOUT	1
STDERR	2

- STDERR can be redirected similarly to STDOUT.

2>

→ all standard error messages should be sent to the file `error.txt`:

```
compsys@compsys-virtualbox: $ ls /fake
ls: cannot access /fake: No such file or directory
compsys@compsys-virtualbox:~$ ls /fake 2> error.txt
compsys@compsys-virtualbox:~$ cat error.txt
ls: cannot access /fake: No such file or directory
```

Redirecting Multiple Streams

File	FD
STDIN	0
STDOUT	1
STDERR	2

`$command 2>&1 filename`

`$command &> filename`

2>&1 redirects STDERR to the same location as STDOUT

`$ls /usr/bin > command.txt 2>&1`

`ls /usr/bin`

#is the command to be executed

`> command.txt`

#redirects the STDOUT of the `ls` command to `command.txt`

`2>&1`

#sends the output of the stderr to the same file as stdout ie `command.txt`

Re-written in a *more streamlined* manner:

`$ls /usr/bin &> command.txt`

2>&1

List Contents of 2 Directories "Documents" & "ABC"
Directory "ABC" not found. "Documents" shown



```
compsys@compsys-virtualbox:~$ ls Documents ABC
ls: cannot access 'ABC': No such file or directory
Documents:
```

Redirecting standard output. Error output is still shown



```
compsys@compsys-virtualbox:~$ ls Documents ABC > dirlist
ls: cannot access 'ABC': No such file or directory
```

```
compsys@compsys-virtualbox:~$ ls Documents ABC > dirlist 2>&1
compsys@compsys-virtualbox:~$ cat dirlist
ls: cannot access 'ABC': No such file or directory
Documents:
```



2>&1

Redirecting error output to standard output

Standard output is already being re-directed to file > dirlist

Hence, both error and standard output are written to file

dirlist

Redirecting Multiple Streams

File	FD
STDIN	0
STDOUT	1
STDERR	2

- Both STDOUT and STDERR can be sent to a file by using the **&>** (this means both **1>** and **2>**)

```
compsys@compsys-virtualbox:~$ ls /fake /etc/ppp
ls: cannot access /fake: No such file or directory
/etc/ppp:
ip-down.d  ip-up.d
compsys@compsys-virtualbox :~$ ls /fake /etc/ppp &> all.txt
compsys@compsys-virtualbox :~$ cat all.txt
ls: cannot access /fake: No such file or directory
/etc/ppp:
ip-down.d
ip-up.d
```

- If you want STDOUT and STDERR to both go to different files:

```
compsys@compsys-virtualbox:~$ ls /fake /etc/ppp > example.txt 2> error.txt
```

/dev/null bitbucket

- /dev/null is the null device
- It takes any input you want and throws it away.
- It can be used to suppress any output

```
compsys@compsys-virtualbox:~$ ls /fake
ls: cannot access '/fake': No such file or directory
compsys@compsys-virtualbox:~$
```

- Control error messages by pushing to /dev/null

```
compsys@compsys-virtualbox:~$ ls /fake 2> /dev/null
compsys@compsys-virtualbox:~$
```



Redirection to Devices

- Can redirect standard output, to not just files, but also devices!

```
$ cat music.mp3 > /dev/audio
```

- The cat command reads the file **music.mp3** and sends the output to **/dev/audio** which is the audio device.

SUMMARY: Redirection Metacharacters

Symbol	Description
>	Directs the standard output of a command to a file. If the file exists, it is overwritten.
>>	Directs the output to a file, adding the output to the end of the existing file.
2>	Directs standard error to the file.
2>>	Directs the standard error to a file, adding the output to the end of the existing file.
&>	Directs standard output and standard error to the file.
<	Directs the contents of a file to the command.
<<	Accepts text on the following lines as standard input.
<>	The specified file is used for both standard input and standard output.